Islamic University of Technology

Course Code : EEE 4308

Course Name : Digital Electronics Lab

Project Name : SAP-1 Architecture-based 8-bit Computer Design and Implementation

Phase : D

Team : Byte_Masters

Name	Student ID
Irfan Hossain Rafi	230021331
Rafin Islam	230021323
Sanajid Arham Bakar	230021311
Raad al Hasan	230021322
Nazifa Noushin	230021117
Samiha Alam Purna	230021127

Supervisor:
Mr. Md. Rahib-Bin-Hossain
Junior Lecturer, Electrical & Electronic Engineering Department
Islamic University of Technology

Objective:

The project's goal is to create an 8-bit computer following the SAP-1 (Simple-As-Possible) architecture by utilizing a combination of combinational and sequential circuits.

Brief Description of Project:

In this project we built an 8-bit SAP-1 in Proteus 8 professional software. Since it is 8-bit we utilized a single 8-bit bus for both address and data transfer for its subsequent operations. A 16x8 bit random access memory (RAM) is integrated for the storage of addresses and 8-bit data, enabling data manipulation and program execution. To manage program execution, the SAP-1 has a program counter that can effectively cycle through addresses from 0000 to 1111 (giving it access to the 16 memory locations). Furthermore, a manual input unit consisting of toggle switches is included to facilitate the initial writing of programs and data into the RAM. The memory address register (MAR) plays a crucial role and used to access data stored in the RAM.

For arithmetic and logic operations, the computer is equipped with an arithmetic and logic unit (ALU) capable of 8-bit addition and subtraction. It employs two registers, the Accumulator (register A) and register B, each with a capacity of 8 bits, for storing data necessary for ALU operations. To ensure the coordinated functioning of all components, instruction register and controller sequence is integrated, which synchronizes the activities of registers, memory, ALU, and other components.

In this phase (Phase D), we built and connected an output register that utilizes multi-digit 7-segment displays to present the final results of the arithmetic operations, allowing us to interpret the numerical output much more easily than binary LEDs. Additional single-digit 7-segment displays are also incorporated into the design to visually indicate current addresses or specific system states during operation. As part of this phase we used an 8-bit bus to integrate all the components mentioned above which we have constructed in previous phases to build a complete SAP-1 for our project.

Submodule:

Output Register:

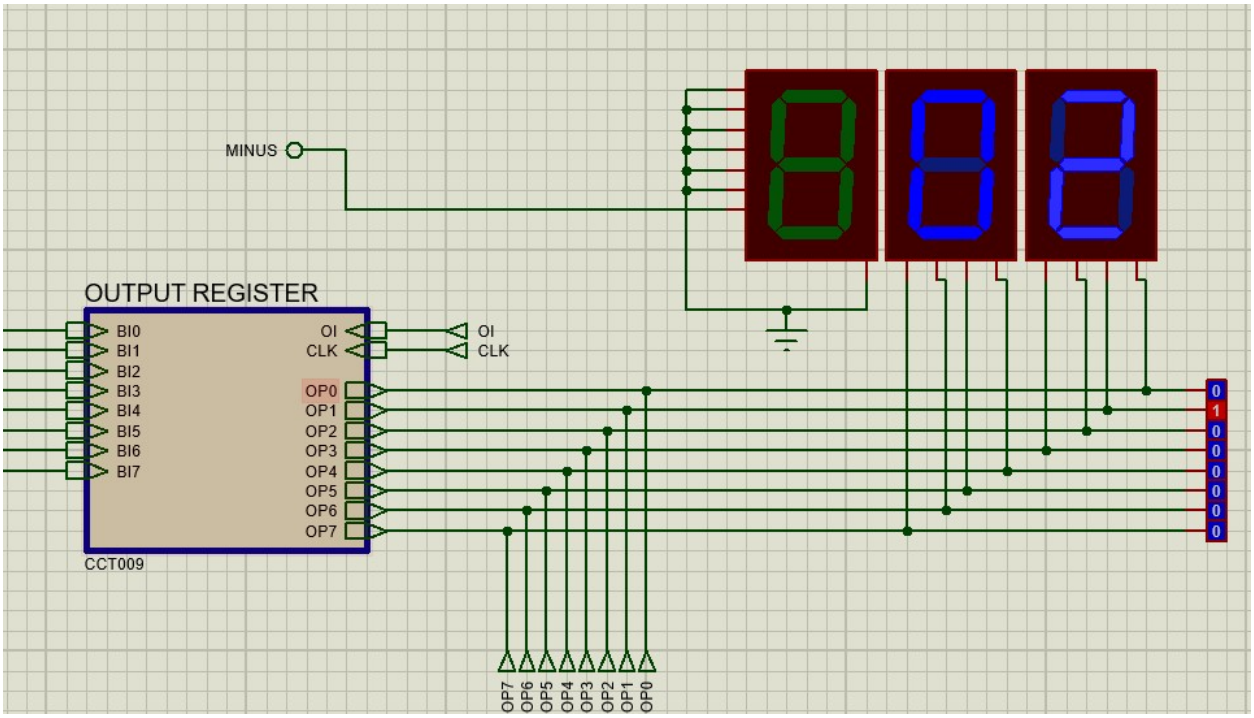


Fig: The Output Register and the multi-digit 7-segment display

The Output Register is where the SAP-1 computer sends processed data. It is used to get and store the final results of calculations so that they can be shown to the user.

The Output Register is an 8-bit register that gets data from the common bus in this system. The Output In (OI) and Clock (CLK) signals control this loading process. The Output Register doesn't put data back on the bus. Instead, its output lines (OP0-OP7) are always sent directly to the display.

The connected multi-digit 7-segment displays turn this 8-bit binary data into a format that is easy for people to read and use. A separate green 7-segment display shows negative results thanks to a separate MINUS signal. This makes sure that the computer's arithmetic operations are shown in full.

IC Components used in Output Register:

74HC173 4-Bit Registers:

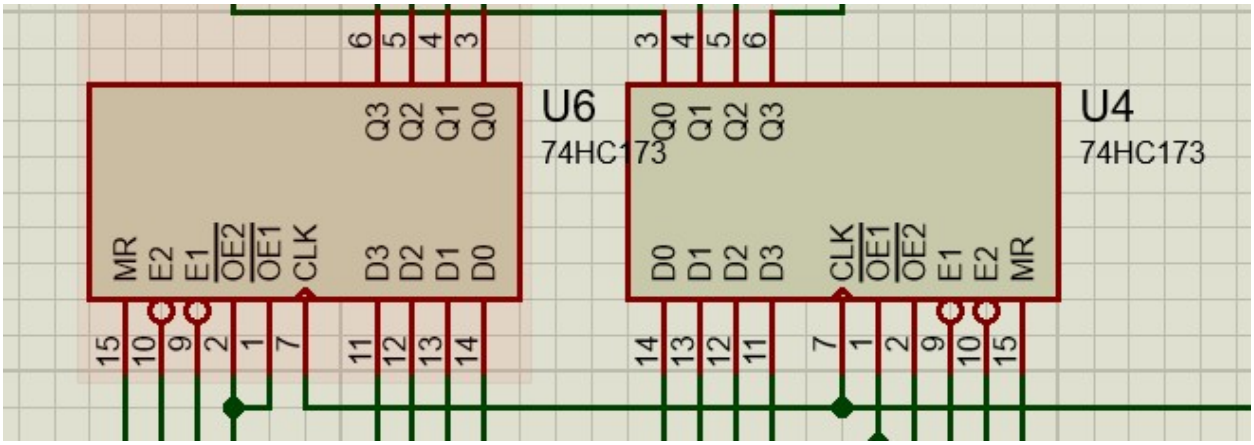


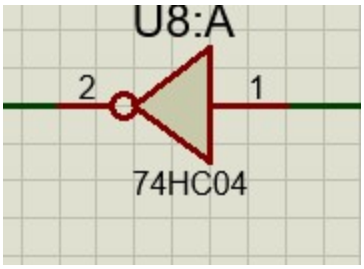
Fig: Block Diagrams of the 4-Bit Registers

Function: Together, these two chips create one 8-bit memory block. U4 stores the lower 4 bits of data, and U6 stores the upper 4 bits.

• Each 74HC173 uses edge-triggered D flip-flops to store 4 bits of data.
• They work on the positive edge of the clock (CLK) signal.
• U4 and U6 make up a full 8-bit output register when they are combined.
• The D inputs get processed data from the system bus through input lines BI0–BI7.
• The data enable pins (E1 and E2) let data be stored on the edge of the clock when the load signal is on and are active low.
• The MR (Master Reset) pins are connected to ground, which turns off the clear function.
• The OE (Output Enable) pins are connected to ground, which keeps the register outputs always available.

• Lines OP0–OP7 send their outputs straight to the 7-segment displays.

74HC04 Inverter:



Function: This chip simply flips the Output In (OI) signal upside down. The register chips (U4 & U6) need a LOW signal to save data, but the computer sends a HIGH signal. The inverter changes the HIGH to a LOW so the registers understand the command.

Description of Connection:

Connections in the Parent Sheet (External)

The input lines (BI0–BI7) connect the Output Register to the main 8-bit system bus so that it can get processed data. The control inputs get the Output In (OI) signal from the control unit and the main system CLK signal. The output lines (OP0–OP7) keep sending the stored 8-bit data straight to the multi-digit 7-segment display modules. A separate MINUS signal goes straight to the leftmost green 7-segment display to show negative numbers. It doesn't go through the register block.

Internal Connections in Child Sheet

The incoming 8-bit bus (BI0–BI7) is split up so that U4 gets the lower 4 bits and U6 gets the upper 4 bits. The outgoing lines (OP0–OP7) take the 4-bit outputs from U4 and U6 and combine them to make the final 8-bit output. The external CLK line goes straight into the clock pins of U4 and U6 to make sure they both store data at the same time. The U8:A inverter sends the external OI line to the data enable pins of U4 and U6. The output enable and master reset pins on both register chips are always connected to the internal ground.

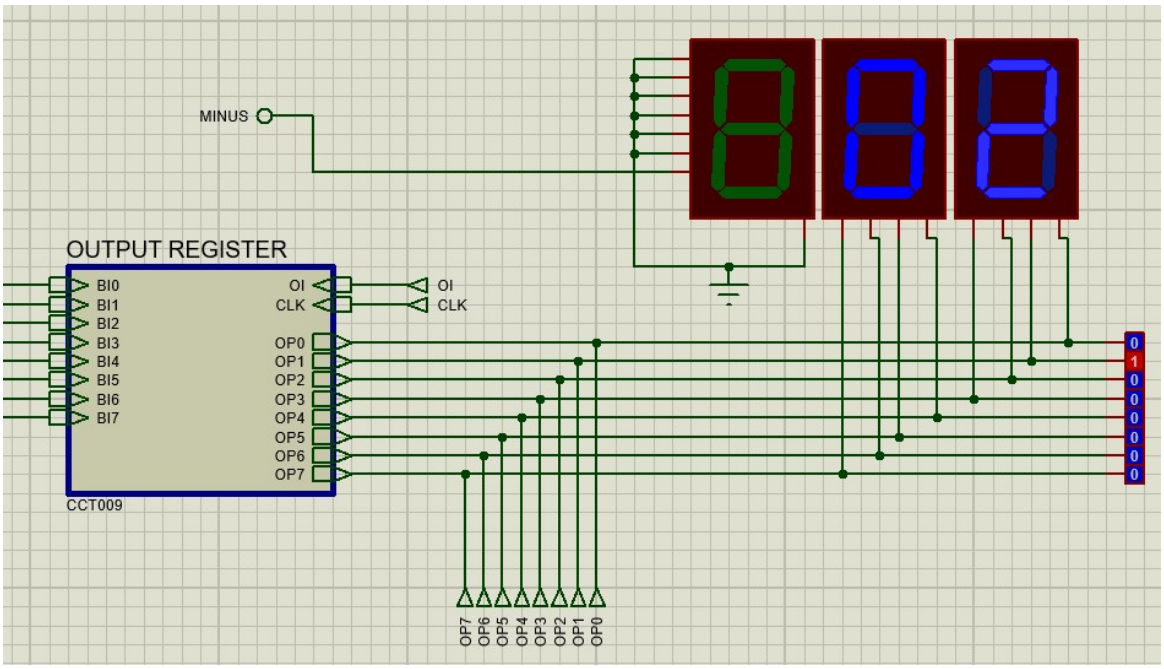

MINUS
OUTPUT REGISTER
BI0
BI1
BI2
BI3
BI4
BI5
BI6
BI7
OI
CLK
OP0
OP1
OP2
OP3
OP4
OP5
OP6
OP7
CCT009
OI
CLK
0
1
0
0
0
0
0
0
OP7
OP6
OP5
OP4
OP3
OP2
OP1
OP0

Fig: Output Register connected to multi-digit 7-segment Decimal Display

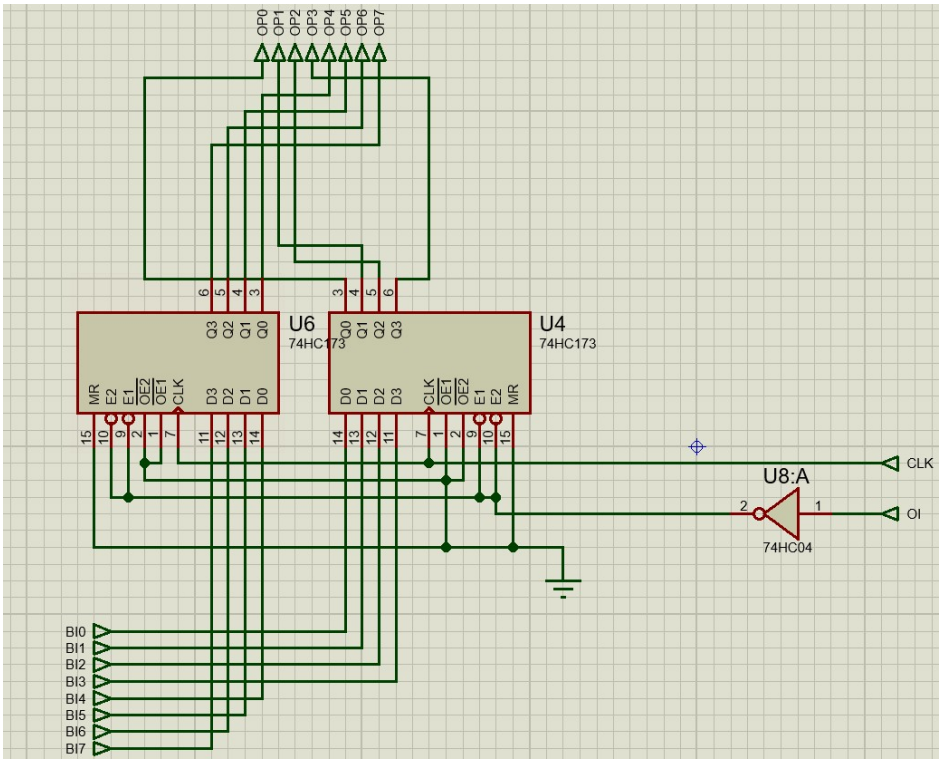


Fig: Child sheet of Output Register

Function:

The Output Register is a storage element in the SAP-1 architecture that acts as the final destination for processed data, allowing users to see the final output of the arithmetic operations. It captures an 8-bit value from the system bus when triggered by the controller sequence's Output In (OI) signal. Once loaded, this value remains stable until a new output command is received. This ensures the final answer is preserved and continuously displayed, giving the user time to read it even as the computer prepares for its next operation.

Unlike internal registers, the Output Register never sends its data back onto the shared system bus. Instead, its output lines are routed directly to the 7-segment displays. This dedicated connection prevents electrical conflicts on the bus while keeping the visual output constantly active. The 7-segment display helps us read the output in the form of decimal numbers rather than binary.

Working Principle:

The operation starts when an 8-bit binary result from a finished calculation is put on the common system bus. The Output Register block's input lines (BI0–BI7) get this data right away. The computer's control unit sends a logical HIGH signal down the Output In (OI) control line to start the saving process when it decides it's time to show this final result.

The 74HC04 inverter chip (U8:A) gets this HIGH OI signal and flips it to a logical LOW. This is because the storage chips need an active-LOW signal to save data. This inverted signal goes into the data enable pins of the two 74HC173 register chips (U4 and U6). This gets them ready to receive the data. At the exact moment that the system clock (CLK) rises, both register chips latch onto the 8 bits of data that are currently waiting at their inputs.

You can see the data right away after it is captured. Because the output enable pins of the registers are always connected to ground, the new data goes straight to the output lines (OP0–OP7) without stopping. This output is always sent to the connected 7-segment display circuitry, which turns the raw binary into a number that people can read. The numbers stay locked and steady on the screen until a new output command starts the whole thing over again.

Bus Integration:

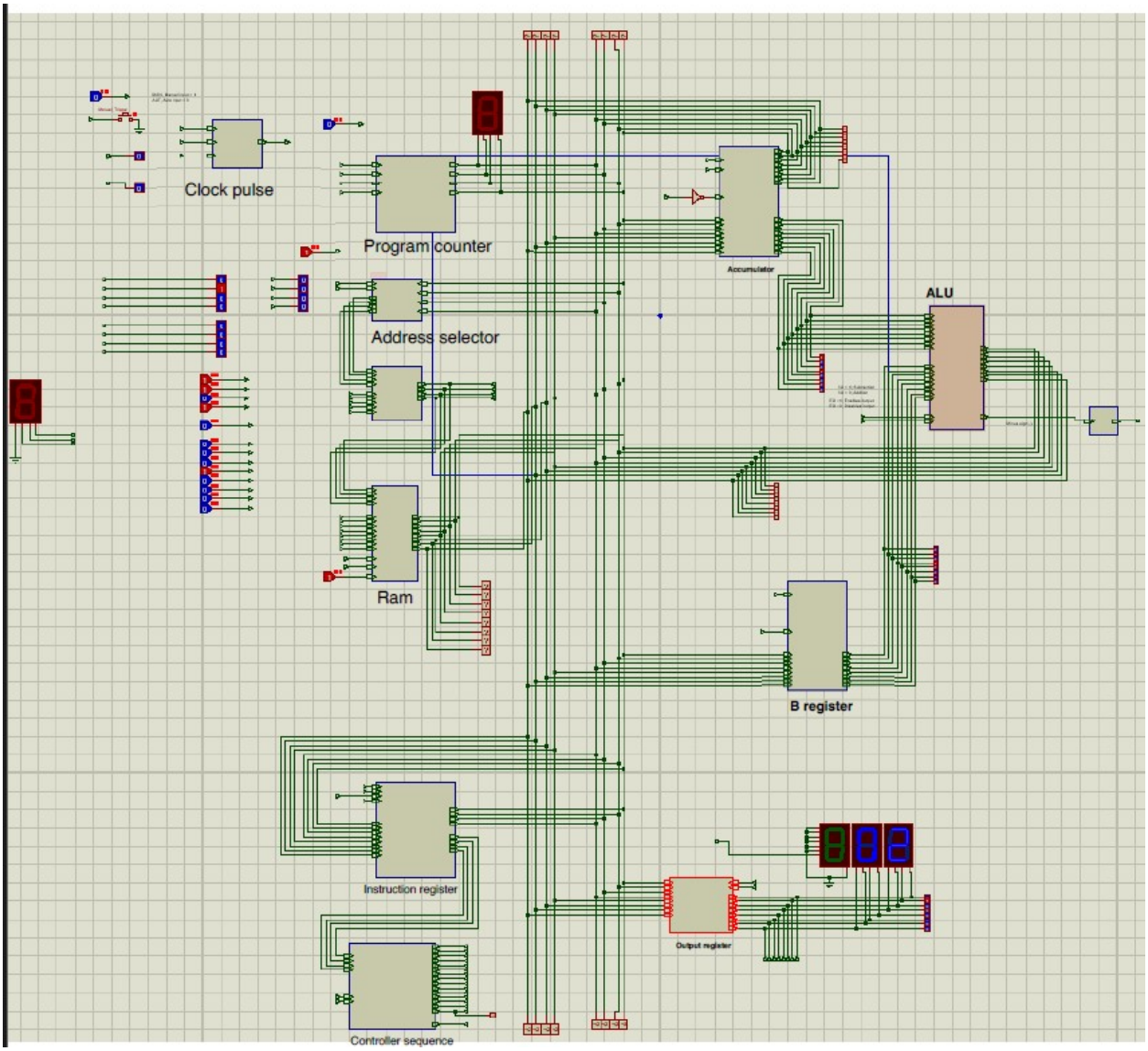


Fig: Bus integration of SAP-1

The bus integration of the SAP-1 (Simple As Possible) computer is the brain of the whole architecture. The system depends on a central communication path, which is shown in the overall schematic by the large vertical wiring that runs down the middle of the circuit.

The 8-bit shared data highway that gets rid of the need for complicated point-to-point wiring between each part. This common backbone, on the other hand, includes all of the main operational blocks. The schematic shows that the following modules are connected to this bus:

Program Counter

Address Selector and RAM

The Instruction Register and the Controller Sequence

Accumulator, ALU, and B Register

Register for Output

The main purpose of this bus integration is to make it easier for the computer to send and receive data in an orderly and synchronized way. Because several outputs are connected to the same wires, so there are chances of contentions. The Controller Sequence controls which module can send data to the bus and which module should read and latch that data while it is being sent.

The bus integration makes sure that these isolated modules can work together without any electrical problems by carefully managing these tri-state connections. This lets them get instructions, do calculations, and show results. In the end, it is this shared bus design that brings all the chips together to make a fully functional, programmable SAP-1.

Function:

The primary function of the SAP-1 bus integration is to act as a centralized, 8-bit communication highway that facilitates synchronized data transfer between all the components such as RAM, ALU otherwise they would have to be connected point-to-point which would make designing the system complex. To prevent electrical contention from multiple modules trying to output data simultaneously, the system relies on the Controller Sequence to manage tri-state connections, dictating exactly which single module can send data onto the bus and which module should read that incoming data during any given clock cycle i.e. synchronization. The bus-integration helps ensure coordination among the components so that the arithmetic operations can be performed and results can be displayed (through the output register).

Working Principle:

Based on the provided schematic, the bus integration acts as the main communication highway for the entire computer, represented by the long vertical green wires running down the center of the circuit. Instead of wiring every single component directly to one another, all the major blocks such as the Program counter, RAM, Accumulator, ALU, and various register connect to this single shared 8-bit pathway. To prevent data from crashing together and causing a short circuit, the Controller sequence block at the bottom acts like a traffic cop. During each beat of the Clock pulse (shown in the top left), the controller sends out specific control signals to ensure that only one module is allowed to push its data onto the bus at a time, while telling another specific module to catch and save that data. By keeping all other components disconnected or quiet during this transfer, the bus allows the computer to safely move instructions and numbers step-by-step to execute a program.

Suppose we want to carry out the operation 5+2-3,
Step 1: Putting in the First Number (LDA 9H)

The Program Counter points to memory address 0H, which is where the instruction LDA 9H (Binary: 0000 1001) is stored.

This instruction tells the computer to load the data from memory address 9H into the Accumulator.

In the data section, address 9H has the value 05H (Binary: 0000 0101). The Accumulator stores this value of 5 as it moves across the W-bus.

Step 2: Add the Second Number (ADD AH)

The Program Counter goes to address 1H and gets the instruction ADD AH (Binary: 0001 1010).

This command tells the computer to get the data at address AH and add it to the value that is already in the Accumulator.

The value at address AH is 02H (in binary, 0000 0010). The B Register gets this value of 2, and the ALU adds it to the 5 that is already in the Accumulator. The Accumulator now has the new total, which is 7.

Step 3: Take away the third number (SUB BH)

The computer gets SUB BH (Binary: 0010 1011) at address 2H. This tells the system to get the data from address BH and take it away from the Accumulator's current total.

Address BH has the value 03H (in binary: 0000 0011). The ALU takes the 7 from the Accumulator, takes away the 3, and gets 4. The Accumulator saves this last result.

Step 4: Getting the Result (OUT)

The instruction at address 3H is OUT (Binary: 0011 0000).

This command makes the Accumulator send its stored value (which is now 4) out onto the W-bus that everyone can see. At the same time, the Output Register is told to read the bus and save the result so that it can be shown to the user.

Step 5: Ending the Program (HLT)

The computer finally sees the HLT instruction (Binary: 1111 0000) at address 0100 (Binary: 4H).

This stops the computer's clock pulses completely, which means the program is done running.

Additional Features:

Doing more than one arithmetic operations in a row: This system can solve longer math problems, such as 5 + 2 - 3. It does this by using the Accumulator. For instance, it saves the answer (7) back into that box after it adds 5 + 2. After that, the computer just takes the next number (3) and subtracts it from the total it saved to get 4. If you have enough memory on your computer, you can keep adding and subtracting like this.

Showing Negative Numbers: We added a separate minus sign light to make the final answers much easier to read. A special memory switch, called a Dual J-K flip-flop, is wired directly into the computer's calculator part (the ALU) and controls this light. The calculator sends a signal to this switch if you do a math problem that goes below zero. The switch takes that signal and turns on the minus sign on the screen. This lets us know whether our results are negative or not.

Discussion:

In Phase D of our project, we put the pieces together to make an 8-bit bus and an output register. We fixed the clock signals and wire connections that were causing problems. This let us run a computer simulation that worked perfectly and was in sync. At first, we had trouble with logic because the main communication bus was sending data signals that were crashing into each other. The Proteus 8 Professional simulation software used wire labels, which is why this happened. Instead, we fixed the problem by making simple, direct wire connections. We also messed up the timing of the system because we forgot to switch the clock signals for some parts at first. Once these issues were fixed, our SAP-1 worked together perfectly. Fixing these timing and contention issues was very important so that the different modules could work together . With the system running smoothly, we were also able to successfully test our extra features, such as chaining multiple math operations together (like 5 + 2 - 3) by using the Accumulator to keep a running total. Furthermore, we verified our custom negative number display, where a Dual J-K flip-flop wired directly to the ALU successfully triggers a minus sign on the screen whenever a calculation drops below zero. For example, it helped the Output Register get the right answer from the cleared 8-bit bus and send it to the seven-segment displays. Finally, these issues were fixed, and the memory, calculator, and output register all work together over the shared bus to carry out commands.

Contribution:

Bus-Integration	Irfan Hossain Rafi, Sanajid Arham Bakar, Nazifa Noushin, Raad Al Hasan, Rafin
Additional Features	Irfan Hossain Rafi
Output Register	Samiha Alam Purna
Report Writing	Sanajid Arham Bakar